Lifting State Up

→ moving state from one component into a parent component.

→ So two or more child component can share it.

Why we need it?

→ Sometime two components need to share or use same data

→ two input that depend on same value

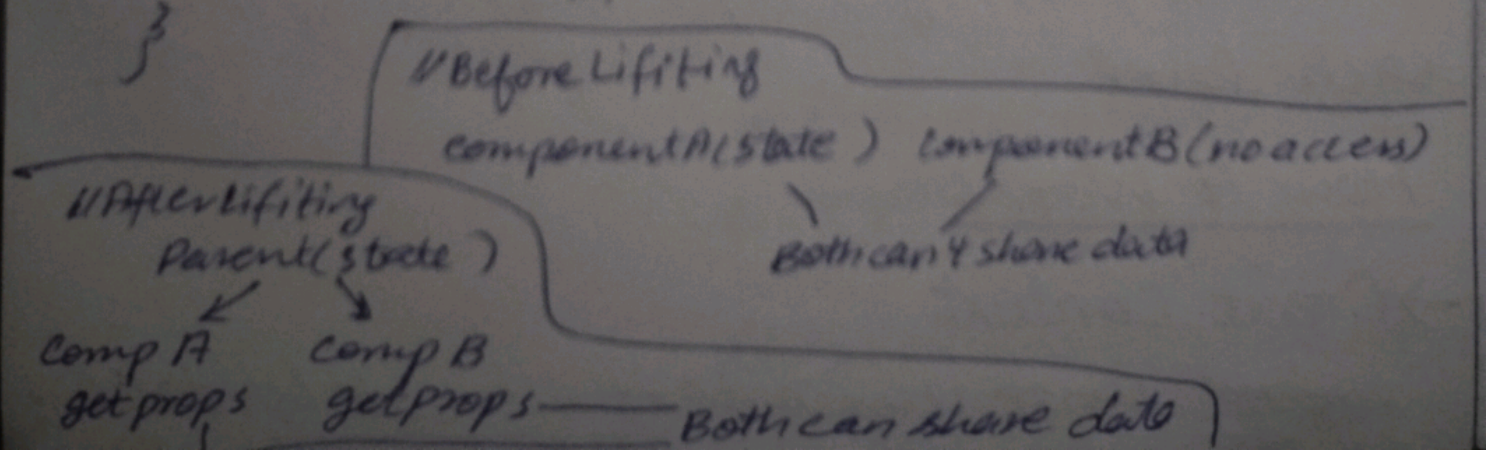
→ A child updating a parent's data

→ A form and a preview showing in same data.

→ if each has own state they stay seperate

```
function childA (){
  const [count, setCount] = useState(0);
  return <button setCount{count+1 ----;
}

function childB(){
  return
    <p>count ???/> can't see childA
}
```



```
// parent holds state
function Parent(){
  const [count, setCount] = useState(0);
  return (
  <div>
     <ChildA count={count} setCount={se...}/>
     <ChildB count = {count}/>
  </div>
);}

// ChildA recieves state and update function
  function childA({count, setCount}){
   return< button onClick ={()=>
        setCount(count + 1)}> Increment</bu>
}

// ChildB recieves State
 function ChildB ({count}){
    return <p> Count: {count}</p>
 }
```

Real-Time Example (Fahrenheit → Celcius)

```
function TemperatureConverter() {
    const [temp, setTemp] = useState(0);
    return (
      <div>
        <CelsiusInput temp={temp} setTemp
                                  ={setTemp} />
        <FahrenheitInput temp={temp} setTemp={setTemp} />
      </div>
    );}

function CelciusInput({ temp, setTemp }){
    return(
      <input
        value={temp}
        onChange={(e) => setTemp(
      />
    );}

function FahrenheitInput({ temp, setTemp })
    {
```

```
return(
  <input
    value={(temp*9)/5+32}
    onChange={(e)=> setTemp(((e.target.value
                              -32)*5)/9)}
  />
);}
```

useContext

→ If you have many nested components, you might have to pass props through several levels:

App → Dashboard → Sidebar → UserInfo

This called prop drilling

Dashboard Sidebar don't need data but pass it down

Solution: useContent

→ It allow you share data globally without passing props manually through each level

How It Works

→ Creat Context

~~export~~ const UserContext = createContext();

Data Flow through Context

Context Provider (holds value)

↓

Component Tree

↓

Any nested component

↓

useContext(MyContext) - Get value directly

Lifting State

→ For 2-3 components

→ Parent-child relationships

→ Explicit (via props)

→ Better for simple cases

→ More performant

→ When to use:

State change frequently

Context

→ For many components

→ Any level of nesting

→ Implicit (via Provider)

→ Better for global data

→ Can cause unnessary re-renders

→ When to use:

- Sharing across many components
- Deep component nesting
- Global data (theme, auth)
- State changes less frequently